

1. RESUMO EXECUTIVO

URLs e Endpoints Oficiais

Site: <https://bundle.social>

API: <https://api.bundle.social>

Docs: <https://docs.bundle.social>

OpenAPI JSON: <https://api.bundle.social/swagger-json>

OpenAPI YAML: <https://api.bundle.social/swagger-yaml>

SDK GitHub: github.com/bundleglobal/bundlesocial-node

Status: bundlesocial.betteruptime.com

Info: info.bundle.social

Contact: contact@bundle.social

Terms: <https://bundle.social/terms>

2. MODELO DE NEGOCIO

Planos

Plano	Preco	Posts/mes
FREE	\$0	20
PRO	\$100/mes	10.000
BUSINESS	\$400/mes	100.000
CUSTOM	Sob consulta	Custom

Plano	Comentarios/mes	Contas sociais
FREE	50	3
PRO	5.000	Ilimitado
BUSINESS	50.000	Ilimitado

Uploads/mes: FREE=200 | PRO=100.000 | BUSINESS=1.000.000

Contagem: Por data de CRIACAO no mes UTC. Rascunhos nao contam. Falhas NAO devolvem cota. Reset dia 1 (UTC).

3. ARQUITETURA TECNICA

3.1 Hierarquia Multi-tenant

- Organization (conta)
- API Keys (compartilhadas entre todos os teams)
 - Webhooks (compartilhados entre todos os teams)
 - Subscription & Billing
 - Team A (ex: "Marketing")
 - Social Accounts (Instagram, TikTok, ...)
 - Posts
 - Uploads
 - Team B (ex: "Cliente: Acme Corp")
 - Social Accounts
 - Posts
 - Uploads

REGRA CRITICA: Rate limits sao por TEAM. Post caps mensais sao por ORGANIZATION.
Para SaaS onde cada cliente precisa suas proprias contas, crie um TEAM por cliente.

3.2 Stack Tecnologico (inferido do scan + OpenAPI)

Componente	Tecnologia	Evidencia
Backend API	Node.js + Express/Nest	SDK TypeScript, OpenAPI 3.0, REST
Linguagem SDK	TypeScript	package.json: typescript ^5.4.5
Geracao cliente	@hey-api/openapi-ts	devDeps
Testes SDK	Jest	jest.config.ts, ts-jest
Release SDK	semantic-release	.releaserc
Storage midia	S3-compativel	Presigned URLs, multipart
Banco dados	PostgreSQL (provavel)	IDs: post_abc, team_xyz, sa_789
Cache/Rate limit	Redis (provavel)	3 camadas, contador por endpoint
Webhooks	Fila assincrona (BullMQ/SQS)	Retries exponenciais, 50 concorrentes
Status page	BetterStack	bundlesocial.betteruptime.com
Frontend	Next.js (confirmado)	_next/static/chunks no scan
WAF/CDN	Cloudflare (confirmado)	Nmap: Cloudflare http proxy
Auth	API Key (x-api-key)	OpenAPI securitySchemes
Spec	OpenAPI 3.0	swagger-json publico
MCP Server	Model Context Protocol	bundlesocial-mcp
CLI	Node.js CLI	bundlesocial-cli, JSON in/out

4. AUTENTICACAO E SEGURANCA

4.1 API Key (cliente -> bundle.social)

Header unico: x-api-key

Prefixo: pk_live_... (padrao Stripe-like)

401 = sem chave | 403 = chave invalida

Chaves sao org-level: uma chave de acesso a todos os teams da organizacao.

Sem OAuth2, sem JWT para o cliente final.

4.2 OAuth (bundle.social -> plataformas sociais)

Fluxo Hosted (recomendado):

1. Cliente chama POST /social-account/create-portal-link
2. bundle.social abre portal que gerencia OAuth UI + selecao de canal + idioma
3. Usuario conclui e e redirecionado de volta

Fluxo Custom UI:

1. Cliente chama POST /social-account/connect (gera OAuth URL)
2. Redireciona usuario para OAuth da plataforma
3. Callback retorna para bundle.social
4. POST /social-account/set-channel (seleciona Page/canal/local)
5. POST /social-account/refresh-channels (atualiza lista)

4.3 Token Management

bundle.social armazena: accessToken, refreshToken, secret, expiresAt

Esses campos NUNCA aparecem em webhooks ou respostas publicas.

Renovacao automatica de tokens.

Deteccao de desconexao remota a cada 6h (Meta: FB, IG, Threads).

POST /social-account/connection-check valida token manualmente.

4.4 Webhook Signature

Header x-signature: HMAC-SHA256

Verificacao com Signing Secret (crypto.timingSafeEqual)

User-Agent enviado: "bundlesocial"

5. ENDPOINTS COMPLETOS (114 total - extraídos da OpenAPI)

5.1 Posts (tag: post)

Endpoint	Metodo	Funcao
/api/v1/post/	GET	Listar posts de um team
/api/v1/post/	POST	Criar post (rascunho/agendado/imediato)
/api/v1/post/{id}	GET	Detalhes de um post
/api/v1/post/{id}	PATCH	Atualizar post (antes de publicar)
/api/v1/post/{id}	DELETE	Deletar post
/api/v1/post/{id}/retry	POST	Re-tentar publicacao de post falho
/api/v1/post/reference-key/{referenceKey}	GET	Buscar post por referenceKey
/api/v1/post/reconnect-social-account	POST	Re-anexar conta a posts orfãos
/api/v1/post/reconnect-social-account/candidates	GET	Listar candidatos a reconexao

5.2 Uploads (tag: upload)

Endpoint	Metodo	Funcao
/api/v1/upload/	GET	Listar uploads
/api/v1/upload/	POST	Simple upload (multipart/form-data, ate
/api/v1/upload/	DELETE	Deletar varios uploads
/api/v1/upload/{id}	GET	Detalhes de upload
/api/v1/upload/{id}	DELETE	Deletar upload
/api/v1/upload/init	POST	Direct upload - presigned URL (ate 5GiB)
/api/v1/upload/finalize	POST	Finalizar direct upload
/api/v1/upload/from-url	POST	Registrar midia de URL publica (ate 1GB)
/api/v1/upload/multipart/init	POST	Iniciar multipart (retorna URLs por part
/api/v1/upload/multipart/sign	POST	Re-assinar partes expiradas
/api/v1/upload/multipart/complete	POST	Juntar partes e registrar
/api/v1/upload/multipart/abort	POST	Cancelar multipart upload

5.3 Social Accounts (tag: socialAccount)

Endpoint	Metodo	Funcao
/api/v1/social-account/by-type	GET	Listar contas por tipo
/api/v1/social-account/connect	POST	Gerar OAuth URL (custom UI)
/api/v1/social-account/create-portal-link	POST	Criar hosted connect link
/api/v1/social-account/connection-check	POST	Validar token manualmente
/api/v1/social-account/copy	POST	Copiar contas entre teams
/api/v1/social-account/disconnect	DELETE	Desconectar conta
/api/v1/social-account/refresh-channels	POST	Atualizar lista de canais
/api/v1/social-account/set-channel	POST	Selecionar canal (Page/canal/local)
/api/v1/social-account/unset-channel	POST	Remover canal selecionado
/api/v1/social-account/profile-refresh	POST	Refresh perfil da conta
/api/v1/social-account/to-delete	GET	Listar contas marcadas para deletar

5.4 Analytics (tag: analytics)

Endpoint	Metodo	Funcao
/api/v1/analytics/post	GET	Analytics de post (normalizado)
/api/v1/analytics/post/raw	GET	Analytics bruto por plataforma
/api/v1/analytics/post/force	POST	Forcar refresh de post
/api/v1/analytics/post/bulk	GET	Analytics em massa (max 60, pag 20)
/api/v1/analytics/social-account	GET	Analytics de conta (normalizado)
/api/v1/analytics/social-account/raw	GET	Analytics bruto de conta
/api/v1/analytics/social-account/force	POST	Forcar refresh de conta

5.5 Comments (tag: comment)

Endpoint	Metodo	Funcao
/api/v1/comment/	GET	Listar comentarios (paginado)
/api/v1/comment/	POST	Criar comentario/resposta
/api/v1/comment/{id}	GET	Detalhes de comentario
/api/v1/comment/{id}	PATCH	Atualizar comentario
/api/v1/comment/{id}	DELETE	Deletar comentario
/api/v1/comment/{id}/retry	POST	Re-tentar comentario falho
/api/v1/comment/import	POST	Iniciar import de comentarios (async)
/api/v1/comment/import	GET	Listar imports de comentarios
/api/v1/comment/import/{importId}	GET	Status de import
/api/v1/comment/import/comments	GET	Comentarios importados (paginado)
/api/v1/comment/import/comments/{commentId}/actor	POST	Acao em comentario importado

5.6 Organization, Team, Import (tags: organization, team, postImport, postCSV)

Endpoint	Metodo	Funcao
/api/v1/	GET	Health check + status por plataforma
/api/v1/organization/	GET	Dados da organizacao
/api/v1/organization/usage/daily-limits	GET	Uso diario por conta
/api/v1/organization/usage/posts	GET	Uso de posts
/api/v1/organization/usage/comments	GET	Uso de comentarios
/api/v1/organization/usage/uploads	GET	Uso de uploads
/api/v1/organization/usage/imports	GET	Uso de imports
/api/v1/team/	GET/POST	Listar/criar teams
/api/v1/team/{id}	GET/PATCH/DELETE	Detalhes/atualizar/deletar team
/api/v1/post-csv-import/	POST/GET	Import via CSV (async)
/api/v1/post-csv-import/{importId}	GET	Status de import CSV
/api/v1/post-csv-import/{importId}/rows	GET	Linhas do CSV
/api/v1/post-csv-import/{importId}/status	GET	Status detalhado
/api/v1/post-history-import/	POST/GET	Importar historico (async)
/api/v1/post-history-import/posts	GET/DELETE	Posts importados
/api/v1/post-history-import/{importId}/retry	POST	Re-tentar import

5.7 Misc Endpoints (tag: misc - operacoes por plataforma)

Endpoints auxiliares por plataforma para moderacao, edicao e features especificas:

Plataforma	Operacoes
Facebook	PATCH/DELETE comment, PATCH/DELETE post, recommendations, token-debug
Instagram	DELETE comment, audio search, locations, tags
TikTok	DELETE comment, CML trending-list
YouTube	PATCH/DELETE comment, video, playlist, thumbnail, regions, categories
LinkedIn	PATCH/DELETE comment, PATCH/DELETE post, reshare, mentions
Twitter/X	DELETE tweet
Threads	(via misc - edicao)
Reddit	PATCH/DELETE comment, PATCH/DELETE post, post-requirements, subreddit-flairs
Pinterest	PATCH/DELETE pin
Mastodon	PATCH/DELETE comment, PATCH/DELETE status
Bluesky	DELETE comment, DELETE post
Discord	DELETE message
Slack	PATCH/DELETE message
Google Business	location, media, reviews, posts, attributes, hours, food-menus

6. SCHEMA COMPLETO DO POST (POST /api/v1/post/)

6.1 Campos Obrigatorios (top-level)

teamId: string (obrigatorio)
title: string, minLength=1 (obrigatorio)
postDate: string ISO 8601 (obrigatorio)
status: "DRAFT" | "SCHEDULED" (obrigatorio)
socialAccountTypes: array, minItems=1 (obrigatorio)
data: objeto com overrides por plataforma (obrigatorio)
referenceKey: string, maxLength=128 (opcional)
firstComment: objeto com primeiro comentario por plataforma (opcional)

6.2 Enum: socialAccountTypes (15 plataformas)

TIKTOK | YOUTUBE | INSTAGRAM | FACEBOOK | TWITTER | THREADS | LINKEDIN | PINTEREST | REDDIT | MASTODON | DISCORD | SLACK |
BLUESKY | GOOGLE_BUSINESS | SNAPCHAT

7. FORMATO DE POST POR PLATAFORMA (data.PLATFORM)

7.1 TWITTER / X (data.TWITTER)

CAMPOS:

- text: string (ate 280 chars)
- uploadIds: array de strings (ate 4 imagens OU 1 video)
- replySettings: enum [EVERYONE, FOLLOWING, MENTIONED_USERS, SUBSCRIBERS, VERIFIED]
- isAiGenerated: boolean (default false) - adiciona label "made with AI" do X

REGRAS:

- Sem analytics via API
- Sem set-channel (OAuth direto)
- Sem comentarios via API
- isAiGenerated obrigatorio se conteudo for AI

7.2 INSTAGRAM (data.INSTAGRAM)

CAMPOS:

- type: enum [POST, REEL, STORY] (default: POST)
- text: string (caption)
- uploadIds: array de strings
- altText: string (texto alternativo para imagem unica)
- thumbnailOffset: number (frame do video como capa, em ms)
- thumbnail: string (URL de imagem no bundle.social)
- shareToFeed: boolean (default true) - Reels: aparece no Feed + Reels ou so Reels
- collaborators: array de strings (usernames)
- autoFitImage: boolean (default false)
- autoCropImage: boolean (default false)
- tagged: array [{username, x(0-1), y(0-1)}] - marca pessoas na foto
- carouselItems: array [{uploadId, altText, tagged[]}] - carrossel multi-imagem
- locationId: string (ID de localizacao do Instagram)
- trialParams: {graduationStrategy: MANUAL|SS_PERFORMANCE} - Reels trial para nao-seguidores
- isPaidPartnership: boolean (default false) - label "Paid partnership"
- brandedContentSponsors: array max 2 usernames (max 30 chars cada)
- musicSoundInfo: {musicSoundId, musicSoundVolume(0-100), videoOriginalSoundVolume(0-100)}
- isAiGenerated: boolean (default false) - label AI do Instagram

REGRAS:

- OAuth via Meta Graph API
- set-channel obrigatorio (selecionar conta IG)
- Reels suportam musica via audio_id
- Aspect ratio 4:5 a 1.91:1
- firstComment maxLength: 2200

7.3 FACEBOOK (data.FACEBOOK)

CAMPOS:

- type: enum [POST, REEL, STORY] (default: POST)
- text: string
- uploadIds: array de strings
- mediaItems: array [{uploadId, altText}] - midia com alt text
- link: string (URL, so para type=POST)
- thumbnail: string (URL imagem no bundle.social)
- mediaTitle: string (titulo para video, so POST com video)
- nativeScheduleTime: ISO 8601 (agendar direto no Meta, max 30 dias futuro)

REGRAS:

- OAuth via Meta Graph API (Page token)
- set-channel obrigatorio (selecionar Page)
- Token expiration e a dor principal
- firstComment maxLength: 8000

7.4 TIKTOK (data.TIKTOK)

CAMPOS:

- type: enum [VIDEO, IMAGE] (default: VIDEO)
- text: string (caption)
- uploadIds: array de strings
- thumbnail: string (URL imagem no bundle.social)
- privacy: enum [SELF_ONLY, PUBLIC_TO_EVERYONE, MUTUAL_FOLLOW_FRIENDS, FOLLOWER_OF_CREATOR]
- photoCoverIndex: integer (default 0, min 0) - capa para posts de foto
- isBrandContent: boolean (default false) - parceria terceiro
- isOrganicBrandContent: boolean (default false) - promover proprio negocio
- disableComments: boolean (default false)
- disableDuet: boolean (default false)
- disableStitch: boolean (default false)
- thumbnailOffset: number (frame capa em ms)
- isAiGenerated: boolean (default false)
- autoAddMusic: boolean (default false) - adiciona musica automatica em fotos
- autoScale: boolean (default false)
- uploadToDraft: boolean (default false) - salvar como rascunho
- musicSoundInfo: {musicSoundId, musicSoundVolume(0-100), musicSoundStart(ms), musicSoundEnd(ms), videoOriginalSoundVolume(0-100)}
musicSoundId = song_clip_id do CML trending list

REGRAS:

- OAuth direto (sem set-channel)
- Comentarios suportados via API
- Analytics suportado
- firstComment maxLength: 150

7.5 YOUTUBE (data.YOUTUBE)

CAMPOS:

- type: enum [VIDEO, SHORT] (default: SHORT)
- uploadIds: array de strings
- text: string (titulo - sempre obrigatorio)
- description: string (descricao do video)
- thumbnail: string (URL imagem no bundle.social)
- privacy: enum [PRIVATE, PUBLIC, UNLISTED]
- defaultLanguage: string (BCP-47, max 35 chars) - idioma titulo/descricao
- defaultAudioLanguage: string (BCP-47, max 35 chars) - idioma audio
- madeForKids: boolean (default false) - OBRIGATORIO
- containsSyntheticMedia: boolean (default false) - conteudo AI
- hasPaidProductPlacement: boolean (default false)

REGRAS:

- OAuth via YouTube Data API v3
- set-channel obrigatorio (selecionar canal)
- Videos ate 4h e 5GB
- 10.000 units/dia (free tier Google)
- firstComment maxLength: 10000
- Playlists suportadas via /misc/youtube/playlist

7.6 LINKEDIN (data.LINKEDIN)

CAMPOS:

- text: string (OBRIGATORIO, ate 3000 chars)
- uploadIds: array de strings
- link: string (URL para article preview)
- thumbnail: string (URL imagem no bundle.social)
- mediaTitle: string (titulo para video/documento)
- privacy: enum [CONNECTIONS, PUBLIC, LOGGED_IN, CONTAINER]
- hideFromFeed: boolean (default false)
- disableReshare: boolean (default false)

REGRAS:

- OAuth via LinkedIn Marketing API
- set-channel obrigatorio (perfil ou Company Page)
- URNs para Company Pages
- firstComment maxLength: 1250
- Mentions suportadas via /misc/linkedin/mentions

7.7 THREADS (data.THREADS)

CAMPOS:

- text: string (ate 500 chars)
- uploadIds: array de strings (ate 10 imagens)
- mediaItems: array [{uploadId, altText}]
- topicTag: string
- replyControl: enum [everyone, accounts_you_follow, mentioned_only, parent_post_author_only, followers_only]
- linkAttachment: string
- poll: {optionA, optionB, optionC?, optionD?} - enquete (min 2 opcoes)
- gif: {gifId, provider: GIPHY}
- allowlistedCountryCodes: array de strings
- crosspostToInstagramStory: boolean
- crosspostToInstagramStoryDarkMode: boolean

REGRAS:

- OAuth via Meta (mesmo ecossistema FB/IG)
- Sem set-channel (OAuth direto)
- firstComment maxLength: 500
- Comentarios suportados
- Analytics suportado

7.8 BLUESKY (data.BLUESKY)

CAMPOS:

- text: string (ate 300 chars)
- uploadIds: array de strings (4 imgs ou 1 video)
- tags: array max 8 (hashtags sem #)
- labels: array enum [!no-unauthenticated, porn, sexual, nudity, graphic-media] - content warnings
- quoteUri: string (AT-URI do post citado, pattern: ^at://S+, max 512)
- externalUrl: string (URI, link card)
- externalTitle: string (titulo do card)
- externalDescription: string (descricao do card)
- thumbnail: string (URI, max 2048) - URL imagem publica no bundle.social
- videoAlt: string (max 10000) - alt text do video

REGRAS:

- OAuth via AT Protocol
- serverUrl opcional (default: https://bsky.social, pode usar PDS/entryway custom)
- Sem set-channel
- firstComment maxLength: 300

7.9 MASTODON (data.MASTODON)

CAMPOS:

- text: string (depende da instancia)
- uploadIds: array de strings
- thumbnail: string (URL imagem no bundle.social)
- privacy: enum [PUBLIC, UNLISTED, PRIVATE, DIRECT]
- spoiler: string (texto de spoiler/content warning)

REGRAS:

- OAuth via Mastodon API (instancia custom)
- serverUrl OBRIGATORIO no /connect (ex: https://mastodon.social)
- Sem set-channel
- firstComment maxLength: 500
- Comentarios suportados
- Analytics basico

7.10 PINTEREST (data.PINTEREST)

CAMPOS:

- text: string
- description: string
- boardName: string (OBRIGATORIO)
- uploadIds: array de strings
- thumbnail: string (URL imagem no bundle.social)
- link: string (URL para qual o Pin linka)
- altText: string (acessibilidade)
- note: string (nota privada, nao visivel publicamente)
- dominantColor: string (cor dominante da imagem)
- isAiGenerated: boolean (default false) - label AI do Pinterest

REGRAS:

- OAuth via Pinterest API v5
- Sem set-channel (boards refresh automatico)
- Sem comentarios via API
- Analytics suportado

7.11 REDDIT (data.REDDIT)

CAMPOS:

- sr: string (OBRIGATORIO) - subreddit name (ex: r/subredditName ou u/username)
- text: string (OBRIGATORIO)
- description: string
- uploadIds: array de strings
- link: string (URL do post link)
- nsfw: boolean (default false)
- flairId: string (obrigatorio se subreddit exigir flair)

REGRAS:

- OAuth via Reddit API
- Sem set-channel
- Comentarios suportados
- Analytics limitado
- firstComment maxLength: 10000
- post-requirements e subreddit-flairs via /misc/reddit

7.12 DISCORD (data.DISCORD)

CAMPOS:

- channelId: string (OBRIGATORIO)
- text: string
- uploadIds: array de strings
- username: string (nome exibido como autor)
- avatarUrl: string (avatar exibido como autor)

REGRAS:

- OAuth via webhook URL
- Sem set-channel
- Comentarios suportados (mensagens)
- Sem analytics
- firstComment maxLength: 2000

7.13 SLACK (data.SLACK)

CAMPOS:

- channelId: string (OBRIGATORIO)
- text: string
- uploadIds: array de strings
- username: string (nome exibido)
- avatarUrl: string (avatar exibido)

REGRAS:

- OAuth via webhook URL
- Sem set-channel
- Comentarios suportados (mensagens)
- Sem analytics
- firstComment maxLength: 30000

7.14 GOOGLE_BUSINESS (data.GOOGLE_BUSINESS)

CAMPOS:

- text: string
- uploadIds: array de strings (imagens/videos)
- topicType: enum [STANDARD, EVENT, OFFER, ALERT] (default: STANDARD)
- languageCode: string (ex: en, en-US)
- callToActionType: enum [BOOK, ORDER, SHOP, LEARN_MORE, SIGN_UP, CALL]
- callToActionUrl: string
- eventTitle: string (se topicType=EVENT)
- eventStartDate: ISO 8601
- eventEndDate: ISO 8601
- offerCouponCode: string (se topicType=OFFER)
- offerRedeemOnlineUrl: string
- offerTermsConditions: string
- alertType: enum [COVID_19] (default: COVID_19, se topicType=ALERT)

REGRAS:

- OAuth via Google My Business API
- set-channel obrigatorio (selecionar local/business)
- Sem comentarios via API
- Analytics suportado
- Reviews suportadas via /misc/google-business/reviews
- Location management completo via /misc/google-business/location

7.15 SNAPCHAT (data.SNAPCHAT)

CAMPOS:

- type: enum [STORY, SPOTLIGHT] (default: STORY)
- uploadIds: array de strings
- text: string
- description: string
- locale: string

- skipSaveToProfile: boolean (default false)

REGRAS:

- OAuth via Snapchat Marketing API (snapkit.com)
- Sem set-channel
- Sem comentarios via API
- Analytics suportado

8. LIMITES DE FIRST COMMENT POR PLATAFORMA

Plataforma	maxLength
TIKTOK	150
YOUTUBE	10000
INSTAGRAM	2200
FACEBOOK	8000
THREADS	500
LINKEDIN	1250
REDDIT	10000
MASTODON	500
DISCORD	2000
SLACK	30000
BLUESKY	300
TWITTER, PINTEREST, GOOGLE_BUSINESS, SNAPCHAT: firstComment nao suportado	

9. SISTEMA DE UPLOAD (3 estrategias)

9.1 Simple Upload (imagens, <90MB)

POST /api/v1/upload/

Content-Type: multipart/form-data

Fields: teamId (string, opcional), file (binary)

Retorna: upload.id

Unico endpoint que usa multipart/form-data

9.2 Direct Upload (ate 5GiB)

1. POST /api/v1/upload/init -> retorna {url, path}

2. PUT <url> --upload-file video.mp4 (direto no storage, presigned URL)

URL presigned expira em 30min

3. POST /api/v1/upload/finalize -> {teamId?, path} -> retorna upload.id

9.3 Multipart Upload (videos grandes, retryable)

1. POST /api/v1/upload/multipart/init -> {uploadId, parts: [{url}]}

Chunks de 64MiB cada

URLs presigned expiram em 6h

2. PUT <part1.url> --upload-file chunk1 (cada parte independente)

3. PUT <part2.url> --upload-file chunk2

4. Se uma parte falhar: POST /api/v1/upload/multipart/sign {partNumbers: [3]}

Re-assina apenas as partes que falharam (partes ja enviadas mantem ETags)

5. POST /api/v1/upload/multipart/complete -> junta partes, retorna upload.id

Enviar ETag de cada parte

6. POST /api/v1/upload/multipart/abort -> cancela (se desistir)

9.4 Upload from URL

POST /api/v1/upload/from-url

Body: {teamId?, url} (URL publica, ate 1GB)

bundle.social faz fetch server-side da URL

Util quando midia ja esta em CDN/S3

10. RATE LIMITS E COTAS

10.1 Rate Limits da API (3 camadas)

Camada	Janela	Max Requests
Burst	1 segundo	100
Short	10 segundos	500
Minute	1 minuto	2000

Implementacao: Contadores por endpoint + por tracker (API key ou IP).

Trafego em POST /post nao consome bucket de GET /analytics.

Resposta 429 quando excedido.

10.2 Limites Diarios por Plataforma (por conta conectada)

Plataforma	FREE	PRO	BUSINESS
Twitter/X	5	15	15
Facebook	10	50	100
Instagram	10	50	100
LinkedIn	10	18	24
YouTube	10	10	15
TikTok	5	10	15
Threads	10	200	250
Pinterest	10	24	36
Reddit	10	24	36
Discord	10	100	200
Slack	10	100	200
Mastodon	10	50	100
Bluesky	10	50	100
Google Business	10	20	40
Snapchat	5	20	40

REGRA CHAVE: A cota segue a CONTA REAL (platform + account id), nao a conexao.

Mesma conta TikTok em 5 teams = 10/day total (nao 50). Contas diferentes = cotas independentes.

10.3 Force Refresh Analytics

Formula: max force refreshes/dia = numero_de_teams x 5

Ex: 10 teams = 50 refreshes/dia. Exceder = 429.

11. WEBHOOKS E EVENTOS

11.1 Eventos Suportados

Evento	Quando dispara
post.published	Post terminou (POSTED ou ERROR)
comment.published	Comentario terminou (POSTED ou ERROR)
social-account.created	Usuario conectou nova conta
social-account.updated	Conta mudou (canal removido, desconexao)
social-account.deleted	Conta desconectada/removida
team.created	Novo team criado
team.updated	Detalhes do team mudaram
team.deleted	Team deletado

11.2 Estrutura do Payload

```
{
  "type": "post.published",
  "data": {
    "id": "post_abc123",
    "status": "POSTED", // ou "ERROR"
    "postDate": "2026-01-15T10:00:00.000Z",
    "postedDate": "2026-01-15T10:00:02.341Z",
    "teamId": "team_xyz",
    "organizationId": "org_123",
    "data": { "INSTAGRAM": { ... } },
    "error": null,
    "errorsVerbose": null,
    "externalData": {
      "INSTAGRAM": {
        "id": "17900000000000000",
        "permalink": "https://instagram.com/p/example/"
      }
    },
    "retryCount": 0,
    "uploads": [...],
    "socialAccounts": [...]
  }
}
```

11.3 Headers Enviados

Header	Valor
Content-Type	application/json
User-Agent	bundlesocial
x-signature	HMAC-SHA256 (verificar com Signing Secret)

11.4 Delivery e Retries

Configuracao	Valor
Timeout	15 segundos por request
Max tentativas	3 (inicial + 2 retries)
Backoff	Exponencial, começando em 30s
Concorrenca	Ate 50 deliveries simultaneos
Auto-disable	7 dias sem sucesso = desativado

12. SISTEMA DE ERROS PADRONIZADO

12.1 Estrutura

```
{
  "errorsVerbose": {
    "INSTAGRAM": {
      "code": "META:190",
      "userFacingMessage": "Seu token expirou. Reconecte.",
      "errorMessage": "OAuthException: Validate permission failed.",
      "isTransient": false
    },
    "TIKTOK": null // Sucesso!
  }
}
```

12.2 Prefixos por Plataforma

Prefixo	Plataforma
META	Instagram, Facebook, Threads
TT	TikTok
LI	LinkedIn
YT	YouTube
HTTP	Erros genericos de API

12.3 Transient vs Non-Transient

Transient (true): Rate limits, timeouts. Acao: retry com backoff exponencial.

Non-Transient (false): Auth errors, validacao. Acao: pedir usuario para corrigir.

12.4 Erros Canonicos Comuns (Meta)

Error key	Causa
account_connection_expired	Token/sessao invalidado
page_or_target_inaccessible	Page nao acessivel
media_fetch_failed	Crawler bloqueado (403)
account_authorization_revoked	App authorization removida
posting_rate_limited	Limite de frequencia/spam
identity_confirmation_required	Confirmar identidade no app
missing_platform_permission	Permissoes de Page revogadas
account_security_limited	Conta limitada por seguranca
platform_login_required	Checkpoint de login

13. DATA RETENTION

Tipo de dado	Retencao
Analytics (parsed + raw)	30 dias
Webhook events	7 dias
Uploads deletados	7 dias apos delecao
Uploads nao usados	Soft-delete 90 dias + 7 dias
Posts importados	30 dias

REGRA DE OURO: Se precisa de historico longo, armazene voce mesmo. Cron job diario que busca analytics e salva no seu banco.

14. BLUEPRINT PARA CONSTRUIR ALTERNATIVA SUPERIOR

14.1 Stack Recomendada

Camada	bundle.social	Recomendado (superior)
Backend	Node.js + Express/Nest	Node.js + NestJS + Fastify
Linguagem	TypeScript	TypeScript (strict mode)
Banco	PostgreSQL	PostgreSQL 16 + Prisma/Drizzle
Cache	Redis	Redis 7 + Upstash (serverless)
Fila	BullMQ/SQS	BullMQ + Redis ou Temporal
Storage	S3-compativel	Cloudflare R2 (zero egress) ou MinIO
Frontend	Next.js	Next.js 15 + App Router + Server Compone
Auth	API Key	API Key + JWT opcional + RBAC granular
Docs	Mintlify/custom	Mintlify ou Fuma (com llms.txt)
Spec	OpenAPI 3.0	OpenAPI 3.1 + Zod schemas
SDK	TypeScript	TS + Python + Go (gerar de OpenAPI)
Monitoring	BetterStack	Grafana + Loki + Prometheus
Deploy	Desconhecido	Docker + Coolify ou Kubernetes
CDN	Cloudflare	Cloudflare (free tier)
MCP	Sim	Sim + A2A (Agent-to-Agent)

14.2 Modelo de Dados PostgreSQL

```
-- Hierarquia: Organization > Team > SocialAccount

CREATE TABLE organizations (
  id TEXT PRIMARY KEY DEFAULT 'org_' || gen_random_uuid(),
  name TEXT NOT NULL,
  plan TEXT DEFAULT 'FREE',
  subscription_status TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE teams (
  id TEXT PRIMARY KEY DEFAULT 'team_' || gen_random_uuid(),
  organization_id TEXT REFERENCES organizations(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  avatar_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE api_keys (
  id TEXT PRIMARY KEY DEFAULT 'key_' || gen_random_uuid(),
  organization_id TEXT REFERENCES organizations(id) ON DELETE CASCADE,
  key_hash TEXT NOT NULL, -- bcrypt/sha256 do pk_live_...
  prefix TEXT, -- pk_live_xxxx (identificacao)
  last_used_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE social_accounts (
  id TEXT PRIMARY KEY DEFAULT 'sa_' || gen_random_uuid(),
  team_id TEXT REFERENCES teams(id) ON DELETE CASCADE,
  type TEXT NOT NULL, -- INSTAGRAM, TIKTOK, etc
  username TEXT, display_name TEXT, external_id TEXT,
  access_token TEXT, -- ENCRYPTADO!
  refresh_token TEXT, -- ENCRYPTADO!
  token_expires_at TIMESTAMPTZ,
```

ANALISE TECNICA COMPLETA - bundle.social

Engenharia Reversa + OpenAPI Real + Scan VulnStrike | Documento confidencial

```
channel_id TEXT, channel_data JSONB,
status TEXT DEFAULT 'ACTIVE',
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE posts (
  id TEXT PRIMARY KEY DEFAULT 'post_' || gen_random_uuid(),
  team_id TEXT REFERENCES teams(id) ON DELETE CASCADE,
  title TEXT, status TEXT DEFAULT 'DRAFT',
  post_date TIMESTAMPTZ, posted_date TIMESTAMPTZ,
  data JSONB NOT NULL, -- { INSTAGRAM: {...}, TIKTOK: {...} }
  external_data JSONB, error TEXT, errors_verbose JSONB,
  retry_count INT DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE uploads (
  id TEXT PRIMARY KEY DEFAULT 'upload_' || gen_random_uuid(),
  team_id TEXT REFERENCES teams(id) ON DELETE CASCADE,
  file_name TEXT NOT NULL, mime_type TEXT NOT NULL,
  size BIGINT, storage_path TEXT NOT NULL,
  status TEXT DEFAULT 'ACTIVE', deleted_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE webhooks (
  id TEXT PRIMARY KEY DEFAULT 'wh_' || gen_random_uuid(),
  organization_id TEXT REFERENCES organizations(id) ON DELETE CASCADE,
  url TEXT NOT NULL, signing_secret TEXT NOT NULL,
  status TEXT DEFAULT 'ACTIVE', last_success_at TIMESTAMPTZ,
  failure_count INT DEFAULT 0, created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE webhook_events (
  id TEXT PRIMARY KEY DEFAULT 'ev_' || gen_random_uuid(),
  webhook_id TEXT REFERENCES webhooks(id) ON DELETE CASCADE,
  type TEXT NOT NULL, payload JSONB NOT NULL,
  status TEXT, response_code INT, attempts INT DEFAULT 0,
  next_retry_at TIMESTAMPTZ, created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE analytics_snapshots (
  id TEXT PRIMARY KEY DEFAULT 'an_' || gen_random_uuid(),
  social_account_id TEXT REFERENCES social_accounts(id) ON DELETE CASCADE,
  post_id TEXT REFERENCES posts(id) ON DELETE CASCADE,
  platform_type TEXT NOT NULL, metrics JSONB NOT NULL,
  raw JSONB, forced BOOLEAN DEFAULT FALSE,
  fetched_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE usage_counters (
  id TEXT PRIMARY KEY DEFAULT 'uc_' || gen_random_uuid(),
  organization_id TEXT REFERENCES organizations(id),
  team_id TEXT REFERENCES teams(id),
  counter_type TEXT NOT NULL, -- posts|comments|uploads|imports
  period TEXT NOT NULL, -- 2026-01 ou 2026-01-15
  used INT DEFAULT 0,
  UNIQUE(team_id, counter_type, period)
);
```

15. PUBLISHING PIPELINE (roteamento)

15.1 Interface PlatformAdapter (padrao Strategy)

```
interface PlatformAdapter {
  getAuthUrl(state: string, options?: OAuthOptions): string;
  handleCallback(code: string): Promise<TokenSet>;
  refreshToken(refreshToken: string): Promise<TokenSet>;
  getChannels(tokenSet: TokenSet): Promise<Channel[]>;
  publish(post: PostData, account: SocialAccount): Promise<PublishResult>;
  getAnalytics(account: SocialAccount, postId?: string): Promise<Analytics>;
  validateConnection(account: SocialAccount): Promise<boolean>;
}

// Implementacoes necessarias:
// - InstagramAdapter (Meta Graph API)
// - FacebookAdapter (Meta Graph API, Page token)
// - TikTokAdapter (TikTok Content API)
// - YouTubeAdapter (YouTube Data API v3)
// - LinkedInAdapter (LinkedIn Marketing API)
// - TwitterAdapter (X API v2)
// - ThreadsAdapter (Threads API)
// - BlueskyAdapter (AT Protocol)
// - MastodonAdapter (Mastodon API, instancia custom)
// - PinterestAdapter (Pinterest API v5)
// - RedditAdapter (Reddit API)
// - DiscordAdapter (webhook URL)
// - SlackAdapter (webhook URL)
// - GoogleBusinessAdapter (Google My Business API)
// - SnapchatAdapter (Snapchat Marketing API)
```

15.2 Rate Limiter (3 camadas com Redis)

```
async function rateLimiter(req, reply, done) {
  const tracker = req.headers['x-api-key'] || req.ip;
  const endpoint = `${req.method}:${req.routeOptions.url}`;
  const now = Date.now();
  const windows = [
    { key: `${tracker}:${endpoint}:1s`, limit: 100, ttl: 1 },
    { key: `${tracker}:${endpoint}:10s`, limit: 500, ttl: 10 },
    { key: `${tracker}:${endpoint}:60s`, limit: 2000, ttl: 60 },
  ];
  for (const w of windows) {
```

```
const count = await redis.incr(w.key);
if (count === 1) await redis.expire(w.key, w.ttl);
if (count > w.limit) {
  return reply.code(429).send({
    error: 'RATE_LIMITED',
    window: w.ttl + 's',
    limit: w.limit
  });
}
}
done();
}
```


16. WEBHOOK DELIVERY WORKER

```
async function webhookWorker(event: WebhookEvent) {
  const webhook = await getWebhook(event.webhookId);
  if (webhook.status === 'DISABLED') return;
  const payload = JSON.stringify({ type: event.type, data: event.payload });
  const signature = hmacSHA256(payload, webhook.signing_secret);
  try {
    const res = await fetch(webhook.url, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'User-Agent': 'yourapp',
        'x-signature': signature
      },
      body: payload,
      signal: AbortSignal.timeout(15000) // 15s timeout
    });
    if (res.ok) {
      await markDelivered(event.id);
      await resetFailureCount(webhook.id);
    } else { throw new Error(`HTTP ${res.status}`); }
  } catch (err) {
    await incrementAttempts(event.id);
    if (event.attempts < 3) {
      const delay = 30 * Math.pow(3, event.attempts - 1);
      await scheduleRetry(event.id, delay);
    } else {
      await markFailed(event.id);
      await incrementFailureCount(webhook.id);
      if (daysSinceLastSuccess(webhook.id) >= 7) {
        await disableWebhook(webhook.id);
        await notifyOwner(webhook.organizationId);
      }
    }
  }
}
```

17. CRON JOBS NECESSARIOS

Job	Frequencia	Funcao
Analytics refresh	A cada 24h	Buscar analytics de todas as contas
Connection check (Meta)	A cada 6h	Validar tokens Meta (FB, IG, Threads)
Token refresh	Proativo	Renovar tokens prestes a expirar
Stale upload cleanup	Diario	Soft-delete uploads nao usados ha 90 dia
Deleted upload purge	Diario	Remover fisicamente uploads deletados ha
Analytics purge	Diario	Deletar analytics com 30 dias
Webhook events purge	Diario	Deletar eventos com 7 dias
Imported posts purge	Diario	Deletar posts importados com 30 dias
Monthly counter reset	Dia 1 (UTC)	Zerar contadores mensais de uso
Webhook auto-disable check	Diario	Desativar webhooks sem sucesso ha 7 dias

18. MELHORIAS PARA SUPERAR BUNDLE.SOCIAL

Area	bundle.social	Sua versao (superior)
Retencao analytics	30 dias (limite duro)	Configuravel: 30/90/365/infinito
Webhook retries	3 tentativas, 30s backoff	5 tentativas + dead-letter queue
Webhook auto-disable	7 dias sem sucesso	Configuravel + alertas antes
SDKs	TypeScript apenas	TS + Python + Go + PHP + Ruby
MCP	Sim (Claude/Cursor)	MCP + A2A + OpenAI function calling
Rate limit headers	Nao documentado	X-RateLimit-Limit/Remaining/Reset
Dashboard	Web only	Web + mobile app (React Native)
Analytics historico	Cliente que se vire	Data warehouse (ClickHouse)
Multi-idioma	Ingles	PT-BR, EN, ES nativo no OAuth
White-label	Menciona mas nao detalha	Full: dominio proprio, logo, cores
Storage	S3 (com egress)	Cloudflare R2 (zero egress) ou MinIO
Observabilidade	BetterStack externo	Grafana + Loki + Prometheus
Pricing	4 tiers fixos	4 tiers + pay-as-you-go + self-hosted
Compliance	Nao mencionado	LGPD/GDPR + data residency
AI agents	MCP server	MCP + webhooks + batch AI posting
Bulk operations	CSV apenas	CSV + JSON + API batch + webhook-trigger
Real-time	Webhooks apenas	WebSocket + SSE + webhooks (3 opcoes)
Self-host	Nao	Docker compose + Coolify

19. PLANO DE EXECUCAO (roadmap)

Fase 1 - Fundacao (semanas 1-3)

1. Setup: Node.js + NestJS + Fastify + Prisma + PostgreSQL + Redis
2. Schema do banco + migracoes
3. Auth: API keys com hash bcrypt, middleware de validacao
4. CRUD: organizations, teams, api_keys
5. Rate limiting (3 camadas com Redis)
6. Health endpoint (/api/v1)

Fase 2 - Plataformas iniciais (semanas 4-7)

1. Interface PlatformAdapter (padrao Strategy)
2. 3 plataformas: Instagram, Facebook, X/Twitter
3. Fluxo OAuth hosted (connect-link) + custom UI
4. Channel selection (set-channel para Meta)
5. Publisher worker (BullMQ) - publica em paralelo
6. Sistema de erros padronizado (code, userFacingMessage, isTransient)

Fase 3 - Midia e publishing (semanas 8-10)

1. Upload simple (multipart/form-data)
2. Upload direct (presigned URL)
3. Upload multipart (chunks de 64MiB, retryable)
4. Upload from-url (fetch server-side)
5. Mais 4 plataformas: TikTok, YouTube, LinkedIn, Threads
6. Scheduler de posts (agenda para postDate)

Fase 4 - Analytics e webhooks (semanas 11-13)

1. Analytics adapter por plataforma (metrics normalizadas)
2. Raw analytics (payload bruto)
3. Force refresh (rate limited: teams x 5)
4. Cron de refresh automatico (24h)
5. Webhooks: cadastro, signing secret, HMAC-SHA256
6. Webhook delivery worker (3 retries, backoff, auto-disable)
7. Eventos: post.published, comment.published, social-account.*, team.*

Fase 5 - Restante + extras (semanas 14-18)

1. 8 plataformas: Pinterest, Reddit, Discord, Slack, Mastodon, Bluesky, Google Business, Snapchat
2. Comments API (importar e criar)
3. Post history import (async job)
4. Bulk post from CSV
5. Link in bio
6. Calendar view no dashboard

Fase 6 - Dashboard + SDK + docs (semanas 19-22)

1. Dashboard Next.js 15: org, teams, contas, posts, calendar, analytics
2. Dark mode + multi-idioma (PT-BR, EN, ES)
3. SDK TypeScript (openapi-ts)
4. SDK Python + Go
5. CLI (JSON in/out)
6. MCP server (Claude/Cursor)
7. Docs (Mintlify + llms.txt + Swagger UI)

Fase 7 - Deploy e operacao (semanas 23-24)

1. Docker compose para self-host
2. Deploy em VPS (Coolify) ou Kubernetes
3. Cloudflare (CDN + WAF + SSL)
4. Storage: Cloudflare R2 (zero egress)
5. Monitoring: Grafana + Loki + Prometheus
6. Status page publico
7. Backup automatico do PostgreSQL

8. Testes E2E + load testing

20. INFRAESTRUTURA E CUSTO

20.1 Custo Estimado Mensal (self-host)

Componente	Opcao	Custo/mes
VPS (backend+workers)	Hetzner CX32 (8vCPU, 16GB)	~\$15
VPS (PostgreSQL)	Hetzner CX22 (4vCPU, 8GB)	~\$10
Redis	Upstash free ou self-host	\$0-10
Storage midia	Cloudflare R2 (10GB free + \$0.015/GB)	~\$0-15
CDN/DNS	Cloudflare free	\$0
Status page	BetterStack free ou Uptime Kuma	\$0
Email transaccional	Resend free (3000/mes)	\$0
Dominio	.com ou .social	~\$1-3
Monitoring	Grafana self-host	\$0
Backup	B2 Backblaze (\$0.006/GB)	~\$2-5
TOTAL		~\$28-58/mes

20.2 API Keys Necessarias por Plataforma

Plataforma	Onde registrar	Custo
Facebook/Instagram	developers.facebook.com	Gratis (review p/ prod)
X/Twitter	developer.x.com	Free limitado; pago p/ volume
TikTok	developers.tiktok.com	Gratis
YouTube	Google Cloud Console	Gratis (10.000 units/dia)
LinkedIn	developer.linkedin.com	Gratis (review Marketing API)
Threads	developers.threads.com	Gratis
Pinterest	developers.pinterest.com	Gratis
Reddit	reddit.com/prefs/apps	Gratis
Discord	discord.com/developers	Gratis (webhook)
Slack	api.slack.com/apps	Gratis (webhook)
Mastodon	Cada instancia ou self-host	Gratis
Bluesky	bsky.social/settings/app-password	Gratis
Google Business	Google Cloud Console (GMB API)	Gratis
Snapchat	snapkit.com	Gratis (review)

21. DADOS DO SCAN VULNSTRIKE (25/08/2026)

21.1 Fingerprinting

Alvo: <https://bundle.social>

WAF: Cloudflare (confirmado via Nmap + fingerprinting)

DBMS: Desconhecido (backend protegido pelo Cloudflare)

Tech Stack: Next.js (confirmado via `_next/static/chunks`)

Server: Cloudflare http proxy

Confianca DBMS: 0.0

21.2 Nmap (40s)

Porta 80/tcp - http Cloudflare http proxy

Porta 443/tcp - http Cloudflare http proxy

Porta 8080/tcp - http Cloudflare http proxy

Porta 8443/tcp - http Cloudflare http proxy

Total: 4 portas abertas

21.3 Spider (28s)

1 URL descoberta (<https://bundle.social>)

0 com parametros | 0 forms | 1 arquivo exposto (robots.txt)

BUG: Browser Spider quebrou (coroutine object has no attribute get)

BUG: Deteccao CMS quebrou (mesmo erro)

Auto-auth: nenhuma pagina de login encontrada

21.4 Fuzzing (19s) - 364 paths

BASELINE 404: 65394 bytes | BASELINE HOME: 401763 bytes

ENDPOINTS ENCONTRADOS (200/307):

- openapi.json (307) - api_docs
- sitemap.xml (200)
- robots.txt (200)
- .well-known/security.txt (200)

403 RESTRITO (10 paths):

- wp-config.php, wp-config.php.bak/.old/.orig
- bootstrap/cache/config.php
- wp-content/uploads, wp-content/debug.log, wp-content/backup
- .aws/credentials, administrator/index.php

FALSOS POSITIVOS FILTRADOS: 52 redirects -> 404

Status codes: {308: 52, 404: 298, 403: 10, 307: 1, 200: 3}

21.5 Active Scan (1s)

0 URLs parametrizadas, 0 forms

1 vuln: Missing Security Headers

- Content-Security-Policy ausente
- X-XSS-Protection ausente
- X-Content-Type-Options ausente
- X-Frame-Options ausente

Payloads injetados em headers ocultos: 0 testes

21.6 JS Secret Scanning (40s)

2 paginas HTML coletadas | 12 scripts JS | 23 arquivos analisados

42 segredos "basic_auth_url" encontrados (FALSO POSITIVO)

Todos em polyfills/webpack do Next.js, nao em codigo do site

Arquivos com "segredos":

1. <https://bundle.social> (raiz)
2. <https://bundle.social/dashboard>
3. `/_next/static/chunks/polyfills-78c92fac7aa8fdd8.js`

4. /_next/static/chunks/71ac0852-8b9b6165fea74bda.js
5. /_next/static/chunks/d562bf09-a3075e53e89fda6f.js
6. /_next/static/chunks/webpack-b6c2bb44fa24ae37.js
7. /_next/static/chunks/main-app-02462e8c6ad6f0f6.js
8. /_next/static/chunks/65708-ae84061b18020e98.js

21.7 Fases que NAO rodaram (scan morreu na fase 8)

BUG em arquivos _exostos.py: cannot access local variable validation
Scan parou na fase vuln_scanner (fase 8 de 15)

NAO RODARAM:

- credential_cascade
- validacao_inteligente
- vuln_scanner_ext (57 tipos)
- nucleis_cve
- sistemas_legados
- graphql_fuzzing
- relatorio_forense

CONCLUSAO: VulnStrike nao consegue lidar com SPA/Next.js + Cloudflare.
So acha falso positivo em site moderno. Feito para sites tradicionais (PHP, WordPress).

22. CHECKLIST DE IMPLEMENTACAO

NOTA FINAL